

SmartCow Tracker

Sistema de Rastreo y Monitoreo de Ganado Bovino

SOFTWARE ARCHITECTURE DOCUMENT (SAD)

IEEE/ISO/IEC 42010:2022 · ISO/IEC 25010:2023 · C4 Model

Versión	1.0.0
Fecha	Mayo 2026
Basado en	IEEE/ISO/IEC 42010:2022
Estado	Borrador - Revisión inicial

Historial de Revisiones

Versión	Fecha	Descripción	Autor
1.0.0	28/05/2026	Versión inicial SAD — SmartCow Tracker	Equipo SmartCow

1. Introducción

1.1 Propósito

El presente Software Architecture Document (SAD) describe la arquitectura de software del sistema SmartCow Tracker, conforme al estándar IEEE/ISO/IEC 42010:2022 (Systems and software engineering — Architecture description). Su propósito es documentar las decisiones arquitectónicas fundamentales, los modelos estructurales y de comportamiento del sistema, las vistas arquitectónicas relevantes para cada stakeholder, y las justificaciones de las decisiones de diseño tomadas.

Este documento sirve como guía autoritativa para el equipo de desarrollo, arquitectos, auditores técnicos y evaluadores de calidad durante todo el ciclo de vida del sistema.

1.2 Alcance

Este SAD cubre la arquitectura completa de la plataforma web SmartCow Tracker, incluyendo: frontend web, backend API, capa de datos, comunicación en tiempo real, integración con dispositivos IoT y módulo de inteligencia artificial. La aplicación móvil nativa queda fuera del alcance de la versión 1.0 de este documento.

1.3 Definiciones y Acrónimos

Término	Definición
SAD	Software Architecture Document. Descripción formal de la arquitectura del sistema.
IEEE 42010	Estándar internacional para descripción de arquitecturas de sistemas y software.
C4 Model	Modelo de 4 niveles para visualizar arquitectura: Contexto, Contenedores, Componentes, Código.
Vista 4+1	Modelo de vistas arquitectónicas: Lógica, Desarrollo, Proceso, Física y Escenarios.
ADR	Architecture Decision Record. Registro de decisión arquitectónica con contexto y consecuencias.
MQTT Broker	Servidor intermediario para el protocolo MQTT que recibe y distribuye mensajes IoT.
ORM	Object-Relational Mapper. Herramienta que abstrae el acceso a base de datos relacional.
CDN	Content Delivery Network. Red de distribución de contenido estático geográficamente distribuida.

SPA	Single Page Application. Aplicación web que carga una sola página HTML y actualiza dinámicamente.
-----	---

1.4 Referencias

- IEEE/ISO/IEC 42010:2022 — Systems and software engineering — Architecture description.
- ISO/IEC 25010:2023 — Systems and software quality requirements and evaluation (SQuaRE).
- C4 Model — Simon Brown (<https://c4model.com>) — Modelo de visualización de arquitectura de software.
- Martin Fowler — Patterns of Enterprise Application Architecture (2002).
- SmartCow Tracker SRS v1.0.0 — Software Requirements Specification (documento anterior).
- OWASP Architecture Security Cheat Sheet (2024).
- The Twelve-Factor App methodology (<https://12factor.net>).

2. Representación Arquitectónica

2.1 Enfoque y Metodología

La arquitectura de SmartCow Tracker se documenta siguiendo el modelo de vistas 4+1 de Philippe Kruchten, complementado con el C4 Model de Simon Brown para la descripción jerárquica de contenedores y componentes. Esta combinación está alineada con los requisitos de IEEE/ISO/IEC 42010:2022, que exige al menos dos puntos de vista (viewpoints) que aborden las preocupaciones de los principales stakeholders.

2.2 Vistas Arquitectónicas Aplicadas

Vista	Modelo / Estándar	Stakeholders que la usan
Lógica	C4 Nivel 1 y 2 (Contexto / Contenedores)	Desarrolladores, Arquitectos, Product Owner
Componentes	C4 Nivel 3 (Componentes por contenedor)	Desarrolladores backend/frontend, Tech Lead
Proceso	Diagramas de flujo y secuencia (UML)	Desarrolladores, QA, Arquitectos
Despliegue	Diagrama de infraestructura cloud / contenedores	DevOps, Operaciones, Auditores de seguridad
Escenarios (+1)	Casos de uso clave / flujos críticos	Todos los stakeholders

3. Objetivos y Restricciones Arquitectónicas

3.1 Objetivos Arquitectónicos

Los objetivos arquitectónicos de SmartCow Tracker derivan directamente de los atributos de calidad definidos en ISO/IEC 25010:2023 y los requisitos no funcionales del SRS. Los principales son:

Atributo de Calidad	Objetivo Arquitectónico	Decisión Arquitectónica Clave
Performance Efficiency	Latencia de actualización GPS <= 2 segundos extremo a extremo.	WebSockets + Redis para estado en tiempo real.
Reliability	Disponibilidad >= 99.5% mensual. RTO < 60 segundos.	Contenedores Docker con health checks y auto-restart.
Security	Zero trust en APIs. Encriptación en tránsito y reposo.	TLS 1.3 + JWT + RBAC + bcrypt + rate limiting.
Maintainability	Cambios en módulos sin afectar otros. Despliegue independiente.	Arquitectura en capas + separación de responsabilidades.
Scalability	Soportar hasta 10,000 dispositivos IoT sin refactoring.	Procesamiento asíncrono MQTT + PostgreSQL particionado.
Portability	Desplegable en AWS, GCP o Railway sin cambios de código.	Contenedores Docker + variables de entorno (12-Factor).

3.2 Restricciones Arquitectónicas

3.2.1 Restricciones Técnicas

- El frontend debe ser una SPA (Single Page Application) compilable como archivos estáticos desplegables en CDN.
- El backend debe exponer una API REST documentada con OpenAPI 3.1 como contrato principal con el frontend.
- La base de datos principal debe ser PostgreSQL 15+ por sus capacidades de datos geoespaciales (PostGIS) y JSON nativo.
- La comunicación en tiempo real entre backend y clientes web debe usar el protocolo WebSocket (RFC 6455).
- Los dispositivos IoT se comunican con el backend exclusivamente via MQTT 3.1.1 sobre TLS.
- Todo el código debe ejecutarse en Node.js 20 LTS para el backend.

3.2.2 Restricciones de Negocio

- El sistema debe ser operativo en zonas rurales con conectividad limitada; el frontend debe funcionar en modo offline.

- El costo de infraestructura inicial no debe superar USD 50/mes para la versión de desarrollo/demo.
- Los datos de la finca A no deben ser accesibles desde la finca B bajo ninguna circunstancia (aislamiento multi-tenant).
- El código fuente debe ser mantenible por un equipo de 1-3 desarrolladores sin arquitectura de microservicios compleja.

4. Vista Lógica — Arquitectura del Sistema (C4 Model)

4.1 Nivel 1 — Diagrama de Contexto del Sistema

El diagrama de contexto muestra SmartCow Tracker como una caja negra y sus relaciones con usuarios externos y sistemas externos. Conforme al C4 Model, este nivel responde a la pregunta: ¿qué construimos y cómo encaja en el mundo?

Descripción del Contexto: SmartCow Tracker recibe datos de dispositivos IoT en campo, los procesa, los almacena y los presenta a tres tipos de actores humanos. Se apoya en servicios externos para mapas, notificaciones y comunicaciones.

Actor / Sistema Externo	Tipo	Interacción con SmartCow Tracker
Administrador / Veterinario / Operador	Usuario humano (Browser Web)	Accede vía HTTPS al dashboard, mapas, alertas y administración.
Collar IoT (ESP32 + GPS)	Dispositivo de hardware	Envía telemetría cada 10s via MQTT/TLS al broker del sistema.
OpenStreetMap / Mapbox	Sistema externo (tiles de mapa)	Provee capas de mapa satelital y callejero al frontend.
SendGrid (Email)	Sistema externo (SaaS)	Recibe solicitudes REST del backend para enviar alertas por email.
Twilio (SMS)	Sistema externo (SaaS)	Recibe solicitudes REST del backend para enviar alertas por SMS.
S3 / Cloud Storage	Sistema externo (nube)	Almacena fotos de animales y archivos de exportación de reportes.

4.2 Nivel 2 — Diagrama de Contenedores

El nivel de contenedores descompone SmartCow Tracker en sus principales piezas de software desplegables e interconectadas. Cada contenedor es un proceso ejecutable o unidad de almacenamiento independiente.

Contenedor	Tecnología	Puerto / Protocolo	Responsabilidad Principal
Web Frontend (SPA)	React 18 + Vite	HTTPS :443	Interfaz de usuario: dashboard, mapas, alertas, administración. Compilado como archivos estáticos.

API Server	Node.js 20 + Express	HTTPS :4000	Lógica de negocio, autenticación JWT, procesamiento de datos, endpoints REST, emisión WebSocket.
MQTT Broker	Mosquitto 2.x	MQTTS :8883	Recibe telemetría de dispositivos IoT. Enruta mensajes al API Server mediante suscripción.
Base de Datos Principal	PostgreSQL 15	TCP :5432	Almacena todos los datos persistentes: ganado, usuarios, alertas, geocercas, historial de posiciones.
Cache / Tiempo Real	Redis 7	TCP :6379	Cache de posiciones GPS más recientes, sesiones activas, pub/sub para WebSocket multi-instancia.
Job Worker	Node.js + Bull MQ	Interno	Procesamiento asíncrono: evaluación de alertas, inferencia de modelo IA, generación de reportes.
Modelo IA / ML	Python + FastAPI	HTTP :8000	Servicio de predicción de salud animal. Expone endpoint REST. Modelo entrenado con TensorFlow/scikit-learn.
Reverse Proxy / CDN	Nginx / Cloudflare	HTTPS :443	Terminación TLS, balanceo de carga, compresión gzip, servir archivos estáticos del frontend.

Nota de diseño: El Job Worker y el API Server comparten la misma base de código pero se despliegan como procesos separados para evitar que trabajos pesados (generación de reportes, inferencia IA) bloqueen el event loop de las peticiones HTTP.

4.3 Nivel 3 — Componentes del API Server

Este nivel detalla los principales componentes internos del contenedor API Server, que es el núcleo de la lógica de negocio del sistema.

Componente	Módulo / Archivo	Responsabilidad
Auth Controller	src/auth/auth.controller.ts	Maneja login, logout, refresh token, 2FA. Genera y valida JWT.
Auth Middleware	src/auth/auth.middleware.ts	Intercepta todas las rutas protegidas, valida JWT, extrae rol y adjunta al contexto de la solicitud.
Animal Router	src/animals/animal.router.ts	Endpoints CRUD de ganado: registro, actualización, búsqueda, importación CSV.

GPS Service	src/gps/gps.service.ts	Procesa coordenadas entrantes de MQTT, actualiza Redis, persiste en PostgreSQL, evalúa geocercas.
Alert Engine	src/alerts/alert.engine.ts	Motor de reglas que evalúa condiciones de alerta contra umbrales configurados. Despacha notificaciones.
WebSocket Gateway	src/realtime/ws.gateway.ts	Gestiona conexiones WebSocket de clientes. Suscribe a Redis pub/sub y emite eventos al cliente correcto.
Geofence Service	src/geofence/geofence.service.ts	Evalúa si las coordenadas GPS de un animal están dentro o fuera de las geocercas activas (algoritmo Point-in-Polygon).
Notification Service	src/notifications/notification.service.ts	Abstracción sobre SendGrid (email) y Twilio (SMS). Envía notificaciones según el nivel de alerta y preferencias del usuario.
Report Service	src/reports/report.service.ts	Genera reportes estadísticos y los exporta en PDF (pdfkit) y XLSX (exceljs). Encola trabajos pesados en Bull MQ.
AI Client	src/ai/ai.client.ts	Cliente HTTP interno que consulta el servicio Python de ML para predicciones de salud animal.
Prisma ORM Client	src/prisma/prisma.service.ts	Capa de acceso a datos. Gestiona el pool de conexiones a PostgreSQL y expone modelos tipados.

4.4 Nivel 3 — Componentes del Frontend (React SPA)

Módulo / Página	Componentes Principales	Responsabilidad	Estado
Dashboard	MetricCards, MiniMap, AlertFeed, CattleQuickAccess	Vista principal con métricas, alertas recientes y mapa resumen.	Redux
MapView	LeafletMap, AnimalMarker, GeofenceLayer, FilterPanel	Mapa interactivo GPS en tiempo real con geocercas y marcadores dinámicos.	Redux + WS
CattleList	SearchBar, FilterChips, CattleCard, ImportModal	Inventario completo del ganado con búsqueda y filtrado en tiempo real.	React Query
CattleDetail	VitalCards, RouteHistory, HealthTimeline, AIPredict	Ficha completa del animal con vitales en tiempo real, historial de rutas y predicciones IA.	Redux + WS

AlertCenter	AlertList, AlertItem, AlertTimeline, Filters	Centro de alertas con gestión del ciclo de vida: reconocer, asignar, cerrar.	React Query
Analytics	LineChart, HeatMap, Histogram, ExportBtn	Dashboard analítico con gráficos interactivos (Recharts) y exportación de reportes.	React Query
Admin Panel	UserManager, GeofenceEditor, DeviceManager, RoleConfig	Administración de usuarios, roles, geocercas y configuración de sensores.	React Query

5. Vista de Proceso — Flujos Clave del Sistema

5.1 Flujo de Telemetría IoT en Tiempo Real

Este flujo describe el recorrido completo de un dato de sensor desde el collar del animal hasta la visualización en el navegador del usuario. Es el flujo más crítico de rendimiento del sistema.

1. El collar IoT (ESP32) captura GPS + temperatura + ritmo cardíaco y empaqueta un payload JSON.
2. El collar publica el mensaje en el topic MQTT: `smartcow/farms/{farmId}/animals/{animalId}/telemetry` sobre TLS :8883.
3. El Broker Mosquitto recibe el mensaje y lo enruta al API Server (que está suscrito al topic).
4. El GPS Service del API Server procesa el mensaje: valida esquema, descripta si aplica, guarda en Redis (posición más reciente) y encola en Bull MQ para persistencia asíncrona en PostgreSQL.
5. Simultáneamente, el GPS Service publica el evento procesado en el canal Redis `pub/sub: realtime:farms:{farmId}`.
6. El WebSocket Gateway, suscrito al canal Redis, recibe el evento y lo emite vía WebSocket a todos los clientes web conectados de esa finca.
7. El frontend (React) recibe el evento, actualiza el estado en Redux y re-renderiza el marcador del animal en el mapa sin recargar la página.
8. En paralelo, el Bull MQ Worker persiste la posición en PostgreSQL (tabla `gps_history`) y evalúa geocercas e umbrales de signos vitales.

Latencia objetivo: El tiempo total entre la transmisión del collar y la visualización en el mapa (pasos 1→7) debe ser ≤ 2 segundos bajo condiciones de red normales (RNF-PE-002).

5.2 Flujo de Generación y Notificación de Alertas

9. El Alert Engine evalúa condiciones: geocerca cruzada, temperatura fuera de umbral, sensor offline, inmovilidad prolongada.
10. Si se detecta una condición de alerta, crea un registro en la tabla `alerts` de PostgreSQL con estado OPEN, nivel de prioridad y metadatos del evento.
11. El Alert Engine publica el evento de alerta en Redis `pub/sub`.
12. El WebSocket Gateway emite el evento de alerta a todos los clientes web de la finca en tiempo real.
13. Según el nivel de prioridad: INFORMATIVO (solo web), ADVERTENCIA (web + email), CRITICO (web + email + SMS).
14. El Notification Service invoca las APIs de SendGrid y/o Twilio de forma asíncrona (no bloqueante).
15. El estado de la notificación (enviada/fallida/reintentada) se registra en la tabla `notification_logs`.

5.3 Flujo de Autenticación y Autorización

16. El usuario ingresa email y contraseña en el frontend.
17. El frontend envía POST /api/auth/login con las credenciales sobre HTTPS.
18. El Auth Controller verifica la contraseña contra el hash bcrypt almacenado. Si falla 5 veces, bloquea la cuenta 15 minutos.
19. Si las credenciales son válidas y 2FA está activo, solicita el código TOTP.
20. Tras autenticación exitosa, genera un JWT (8h) y un refresh token (30d, almacenado en cookie HttpOnly).
21. El JWT contiene: userId, farmId, role y permisos específicos.
22. En cada solicitud posterior, el Auth Middleware valida el JWT, extrae el contexto y aplica RBAC antes de pasar la solicitud al controlador.

6. Vista de Despliegue — Infraestructura y Topología

6.1 Entornos del Sistema

Entorno	Propósito	Infraestructura	URL / Acceso
Development	Desarrollo local del equipo.	Docker Compose local. Base de datos local.	localhost:5173 / :4000
Staging	Pruebas de integración y UAT.	Railway / Render. BD en Supabase staging.	staging.smartcow.app
Production	Entorno productivo de usuarios reales.	Railway o AWS ECS. BD en Supabase/RDS. CDN Cloudflare.	app.smartcow.app

6.2 Topología de Despliegue en Producción

La topología de producción está diseñada para máxima disponibilidad con mínimo costo operacional inicial, siguiendo la metodología de Twelve-Factor App para facilitar el escalado horizontal futuro.

Nodo / Servicio	Tecnología de Hosting	Configuración de Producción
Frontend SPA	Vercel / Cloudflare Pages	Build estático con Vite. CDN global. Cache-Control: max-age=31536000 para assets hash.
API Server	Railway / AWS ECS (Docker)	Contenedor Node.js 20-alpine. 2 réplicas mínimo. Health check en /health cada 30s.
MQTT Broker	Railway / EC2 t3.small	Mosquitto 2.x con TLS. Certificados Let's Encrypt renovación automática.
PostgreSQL	Supabase / AWS RDS	PostgreSQL 15 con PostGIS. Backup diario automático. Connection pooling con PgBouncer.
Redis	Upstash Redis / ElastiCache	Redis 7 con persistencia AOF. Usado para cache, sesiones y pub/sub de tiempo real.
AI Service	Railway / AWS ECS (Docker)	Contenedor Python 3.11-slim + FastAPI + scikit-learn. Escalado bajo demanda.
Reverse Proxy	Nginx + Cloudflare	Nginx como reverse proxy interno. Cloudflare como WAF y CDN externo con DDoS protection.

6.3 Estrategia de Contenedores Docker

Todos los servicios del backend se empaquetan como imágenes Docker para garantizar reproducibilidad en todos los entornos. La estructura de imágenes es:

- smartcow-api: Node.js 20-alpine. Multi-stage build. Imagen final < 200MB.
- smartcow-worker: Misma base que api. Diferente entrypoint (worker.js en lugar de server.js).
- smartcow-ai: Python 3.11-slim + FastAPI + modelo ML serializado. Imagen < 500MB.
- smartcow-mqtt: Mosquitto 2.x oficial con configuración TLS montada como volumen.

La orquestación local usa docker-compose.yml con todos los servicios. En producción se usa el orquestador nativo de la plataforma cloud (Railway services o ECS tasks).

6.4 Estrategia de Seguridad en Red

- TLS 1.3 en todas las comunicaciones externas (frontend ↔ API, IoT ↔ MQTT Broker).
- La base de datos PostgreSQL y Redis NO tienen exposición pública; solo son accesibles desde la red privada interna del proveedor cloud.
- El API Server se comunica con PostgreSQL y Redis exclusivamente por red privada (VPC/VNET).
- El MQTT Broker acepta conexiones solo en el puerto 8883 (MQTTS), con autenticación por certificado de cliente para los dispositivos IoT.
- Cloudflare actúa como WAF externo: protege contra DDoS, inyecciones y bots antes de que el tráfico llegue al servidor.
- Rate limiting implementado en Nginx (capa de red) y en el API Server (capa de aplicación) como defensa en profundidad.

7. Decisiones Arquitectónicas Clave

Esta sección resume las decisiones de diseño más importantes. El detalle completo de cada decisión se documenta en los Architecture Decision Records (ADR), que es el siguiente documento en la serie de documentación técnica de SmartCow Tracker.

ID	Decisión	Justificación	Consecuencias
DA-01	Arquitectura en capas (Layered) para el backend sobre microservicios.	Equipo pequeño (1-3 devs). Microservicios añaden complejidad operacional innecesaria en esta etapa. La escalabilidad requerida se logra con monolito modular + contenedores.	Menor complejidad inicial. Mayor acoplamiento a largo plazo; se mitiga con interfaces bien definidas entre módulos.
DA-02	PostgreSQL como única base de datos relacional (no MongoDB o multiDB).	PostgreSQL soporta datos geospaciales (PostGIS para geocercas), JSON nativo (JSONB para telemetría flexible), arrays nativos y transacciones ACID.	Mayor complejidad en esquema inicial. Se mitiga con Prisma ORM y migraciones automatizadas.
DA-03	Redis como capa de estado en tiempo real (no polling a PostgreSQL).	Las posiciones GPS más recientes de 247+ animales necesitan lectura en < 50ms. PostgreSQL con esa carga de escritura/lectura concurrente generaría latencias inaceptables. Redis con estructuras HASH es la solución óptima.	Dependencia adicional de infraestructura. Datos en Redis son efímeros (no fuente de verdad). Requiere sincronización Redis-PostgreSQL vía worker.
DA-04	MQTT sobre HTTP para comunicación con dispositivos IoT.	MQTT está diseñado para IoT: QoS configurable, bajo overhead de protocolo, retención de mensajes y soporte para redes inestables con keep-alive. HTTP polling generaría 60x más tráfico con la misma frecuencia de actualización.	Requiere un broker MQTT adicional (Mosquitto). Añade complejidad de configuración TLS para el hardware.
DA-05	React 18 + Vite como stack frontend (no Next.js SSR).	SmartCow Tracker es una aplicación de datos en tiempo real, no un sitio de contenido estático. SSR no aporta beneficios de SEO ni de performance de carga inicial para una aplicación tras login. Vite ofrece HMR instantáneo y builds más rápidos.	Sin SEO en páginas internas (no es requerimiento). Sin ventajas de caché de servidor. Frontend servible desde CDN sin servidor de aplicación.

DA-06	Servicio de IA como microservicio Python independiente (no integrado en Node.js).	El ecosistema de ML en Python (TensorFlow, scikit-learn, pandas) no tiene equivalentes maduros en Node.js. Mantenerlos en contenedores separados permite escalar el servicio IA independientemente y reemplazar el modelo sin afectar el API Server.	Latencia de red adicional en llamadas internas. Requiere gestión de dos stacks tecnológicos. Se mitiga con comunicación HTTP interna en la misma VPC.
-------	---	--	---

8. Escenarios de Atributos de Calidad

Conforme a IEEE/ISO/IEC 42010:2022, las decisiones arquitectónicas deben justificarse en términos de su impacto en los atributos de calidad del sistema. Los siguientes escenarios describen situaciones concretas y cómo la arquitectura los resuelve.

8.1 Escenario de Rendimiento — Actualización GPS Masiva

Elemento	Descripción
Estímulo	247 collares IoT transmiten posición GPS simultáneamente cada 10 segundos (pico de 25 mensajes/segundo).
Fuente	Dispositivos IoT en campo durante horario de pastoreo pico (6am-8am).
Respuesta arquitectónica	MQTT Broker absorbe el pico. GPS Service actualiza Redis de forma síncrona (< 5ms). La persistencia en PostgreSQL se delega al Bull MQ Worker asincrónamente. WebSocket emite a clientes conectados vía Redis pub/sub. El frontend renderiza solo los marcadores con posición cambiada (React diff + memoización).
Resultado medible	Latencia total dispositivo → visualización <= 2 segundos. CPU del API Server < 40% bajo esta carga.

8.2 Escenario de Seguridad — Intento de Acceso No Autorizado

Elemento	Descripción
Estímulo	Un actor malicioso intenta acceder a los datos de la Finca A usando credenciales de la Finca B o un token JWT manipulado.
Respuesta arquitectónica	El Auth Middleware valida la firma del JWT con la clave secreta del servidor. Si el farmId en el token no coincide con el recurso solicitado, devuelve HTTP 403. Todas las queries de Prisma incluyen filtro por farmId extraído del JWT. Es imposible acceder a datos de otra finca sin un JWT válido de esa finca.
Resultado medible	100% de los intentos de acceso cross-tenant son rechazados con HTTP 403 y registrados en el log de auditoría.

8.3 Escenario de Fiabilidad — Falla del API Server

Elemento	Descripción
Estímulo	El contenedor del API Server falla por error de memoria o excepción no manejada.

Respuesta arquitectónica	El orquestador de contenedores (Railway/ECS) detecta el fallo via health check en /health (polling cada 30s). Reinicia el contenedor automáticamente. La segunda réplica del API Server continúa atendiendo solicitudes durante el reinicio. Redis mantiene el estado de sesiones y posiciones GPS sin pérdida.
Resultado medible	Tiempo de recuperación (RTO) < 60 segundos. Disponibilidad >= 99.5% mensual. Los clientes web reconectan automáticamente via WebSocket con backoff exponencial.

Apéndice A: Estructura de Directorios del Proyecto

La siguiente estructura de directorios refleja la arquitectura en capas del backend y la organización por funcionalidades (feature-based) del frontend.

Backend (Node.js + Express + Prisma):

```
smartcow-api/
├── src/
│   ├── auth/           # Autenticación, JWT, 2FA
│   ├── animals/        # CRUD ganado, importación CSV
│   ├── gps/            # Procesamiento telemetría IoT
│   ├── geofence/       # Geocercas, Point-in-Polygon
│   ├── alerts/         # Motor de alertas y reglas
│   ├── realtime/       # WebSocket Gateway
│   ├── notifications/  # Email (SendGrid) + SMS (Twilio)
│   ├── reports/        # Generación PDF/XLSX
│   ├── ai/             # Cliente HTTP servicio ML Python
│   ├── prisma/         # ORM + conexión PostgreSQL
│   ├── redis/          # Cache + pub/sub
│   ├── common/         # Middlewares, guards, pipes, utils
│   └── app.ts          # Entry point Express
├── prisma/schema.prisma # Esquema de base de datos
├── Dockerfile
└── docker-compose.yml
```

Frontend (React 18 + Vite + Tailwind):

```
smartcow-web/
├── src/
│   ├── features/       # Módulos por funcionalidad
│   │   ├── dashboard/
│   │   ├── map/
│   │   ├── cattle/
│   │   ├── alerts/
│   │   ├── analytics/
│   │   └── admin/
│   ├── components/    # Componentes reutilizables
│   ├── store/         # Redux Toolkit (estado global)
│   ├── hooks/         # Custom hooks (useWebSocket, etc)
│   ├── services/      # Axios API clients
│   └── utils/
```

Apéndice B: Tecnologías y Versiones del Stack

Categoría	Tecnología	Versión	Propósito
-----------	------------	---------	-----------

Frontend	React	18.3	Biblioteca de UI con hooks y context.
Frontend	Vite	5.x	Bundler y servidor de desarrollo con HMR.
Frontend	Tailwind CSS	3.4	Framework de estilos utility-first.
Frontend	shadcn/ui	latest	Componentes UI accesibles sobre Radix UI.
Frontend	Leaflet.js	1.9	Mapas interactivos con tiles OpenStreetMap.
Frontend	Redux Toolkit	2.x	Gestión de estado global (GPS en tiempo real).
Frontend	React Query	5.x	Cache y sincronización de datos del servidor.
Frontend	Recharts	2.x	Gráficos analíticos interactivos.
Backend	Node.js	20 LTS	Runtime JavaScript del servidor.
Backend	Express	4.x	Framework HTTP minimalista para API REST.
Backend	Prisma ORM	5.x	ORM tipado con migraciones automáticas.
Backend	Socket.IO	4.x	WebSockets con fallback y rooms.
Backend	Bull MQ	5.x	Cola de trabajos asíncronos sobre Redis.
Backend	MQTT.js	5.x	Cliente MQTT para suscripción al broker.
Base de Datos	PostgreSQL	15	BD relacional principal con PostGIS.
Cache	Redis	7.x	Cache, sesiones y pub/sub tiempo real.
IoT Broker	Mosquitto	2.x	Broker MQTT para dispositivos de campo.
IA / ML	Python + FastAPI	3.11 / 0.110	Servicio de predicción de salud animal.
Infraestructura	Docker	24+	Contenedores para todos los servicios.

— Fin del Documento SAD v1.0.0 —

SmartCow Tracker | Documento confidencial | Mayo 2026